The .ADS Firmware Container

Format Specification and Reference Decoder

Community Reverse-Engineering Document
Jared Cabot • with Claude (Anthropic)
Revision 1.1 • August 2026

Scope

This document specifies the on-disk format of the SIGLENT .ADS firmware update file used by the SDG2000X series, completely: the container framing, the obfuscation transform, the two-key triple-DES layer, the member payloads, and the integrity checks. It gives a reference decoder and a full account of how the format was recovered and verified. It is derived from a single release, `SDG2000X_P39R7.ADS`, and cross-checked against the instrument's own application binary.

Provenance

Every structural claim is traced to one of two primary sources: the bytes of the .ADS file itself, and the routines in the instrument's application binary that read it. Where a function is named it is the name in the binary's dynamic symbol table. The non-standard cipher was independently cross-checked against the long-running EEVblog community thread on this format and against analogNewbie's reference implementation posted there (pyDesSiglent.py); the two agree byte-for-byte. Nothing here is guessed.

Applicability

The format is shared across the SDG2000X hardware generations. One .ADS carries images for both the Xilinx Zynq-7000 hardware and the earlier Texas Instruments Sitara AM3359 hardware. The transform, keys and framing described here apply to both; only the member payloads differ.

Conventions

Hexadecimal takes a `$` prefix. Byte offsets are decimal or hexadecimal as convenient and always absolute unless a base is named. `this_type` denotes a function in the application binary. Bit and byte order is little-endian throughout, matching the ARM target.

Disclaimer

This is not a SIGLENT publication and carries no endorsement from SIGLENT Technologies. It documents an update format for the purpose of inspection and recovery. The triple-DES key given here is a fixed constant embedded in shipping firmware; it is reproduced because it is necessary to read the format and is not secret in any meaningful sense. Building or installing a modified .ADS risks rendering an instrument unbootable with no remote recovery path.

Contents

1

Overview

What a .ADS File Is

A `.ADS` file is the firmware update package for the SIGLENT SDG2000X. The instrument reads it from a USB drive, unpacks it, writes the results to its boot and root partitions, and reboots. The release documented here, `SDG2000X_P39R7.ADS`, is 40,525,383 bytes and is dated 5 January 2026.

Opened in a hex editor the file looks like ciphertext: its byte histogram is flat, it carries no recognisable magic number at any offset, and a signature scanner finds nothing. That appearance is the result of a light obfuscation over ordinary compressed data, not of encryption of the whole file. Most of the package is recovered by undoing three reversible transforms. A small part of it, and only a small part, is genuinely encrypted with triple-DES; the key for that is a fixed constant in the instrument's own firmware and is given in Chapter 4.

The Four Layers

The format is best understood as four layers, outermost first. This document takes them in the order a decoder unwinds them.

Table 1-1. The Layers of a .ADS File

Layer	What it is	Chapter
Header	A 112-byte encrypted header carrying a CRC, the decoded length, a product id and vendor tags.	2
Cipher	A non-standard two-key triple-DES over two small regions of the payload.	4
Obfuscation	A whole-payload byte reversal and two exclusive-OR masks.	3
Container	A short record header, then a genuine ZIP archive (members, central directory and EOCD).	2, 5

The layering is not strictly nested. The header is a separate 112-byte read. The cipher and the obfuscation both operate on the payload that follows it, the cipher first and the obfuscation second, and the container structure only becomes visible once both are undone.

How the Instrument Reads It

The application binary drives the update from `dev_update_system`. That routine reads the 112-byte header, decrypts it, and checks the product and version. It then calls `dev_upgrade_detail_acitve`, which reads the rest of the file, verifies a checksum, calls `dev_decode_data` to decrypt the two cipher regions and undo the obfuscation, and finally calls `dev_fw_update_ads` to split the container into its members and unzip them into place.

Every routine named in this document is a real, named function in the shipping binary. The binary is stripped of its ordinary symbol table but retains a dynamic symbol table of 40,601 defined symbols, so each step can be followed to a named function and read directly rather than inferred. The build path compiled into the binary, `/home/brus_peng/workdir/sdg2000x_zynq/trunk/`, shows it is one product of a source tree that serves the whole SDG line.

2

Container Structure

File Layout

At the coarsest level the file is a 112-byte header followed by a payload of 40,525,271 bytes. The header is consumed separately; everything else is the payload, and all the transforms in Chapters 3 and 4 operate on it.

Table 2-1. Top-Level File Layout

Offset	Length	Contents
$0000	112	Encrypted header (Chapter 2)
$0070	40,525,271	Payload: cipher + obfuscation + container

The Encrypted Header

The first 112 bytes are read into their own buffer and passed to `dev_decode_header_data`, which decrypts them with the same cipher and key as the payload regions. The header is not obfuscated: unlike the payload it is neither reversed nor masked, only decrypted. Decrypted, it is a structured record whose fields are given in Table 2-2. The instrument uses it for a pre-flight check, reading a vendor tag and a product code before committing to the update.

Table 2-2. The Decrypted 112-Byte Header

Offset	Type	Value in P39R7	Meaning
$00	u32	$CC677E5E	CRC of the file
$04	u32	$026A5DD7	Decoded payload length (= 40,525,271)
$0C	u32	$00002968	Product id (10600)
$26	char[]	`SIGLENT`	Vendor tag (NUL-terminated)
$3A	char[]	`ISP1763`	USB host-controller tag
other	u8[]	0	Zero padding

The length field at $04 equals the decoded payload length exactly, in every release examined, which makes the decrypted header a useful cross-check on a decode. The header is not part of the container and is not needed to recover the members, so a decoder that only wants the payload may discard it.

Note

What matters for decoding is that the payload begins at file offset 112, not at zero. A decoder that reverses the whole file rather than the payload will misplace every member.

The Container Record

Once the payload is decrypted and de-obfuscated (Chapters 3 and 4), its first 52 bytes ($34) are a small record header, and the members follow at offset $34. The header has three used fields and is otherwise zero.

Table 2-3. The Container Record at Payload Offset $00

Offset	Type	Value in P39R7	Meaning
$00	u32	$C3F3F2A3	Checksum of the member region (see Chapter 5)
$04	u32	$026A5DA3	Length of the member region: 40,525,219 = payload – 52
$08	u8	7	Type code
$09+	u8[]	0	Reserved, zero to $33

The length field is exactly the payload length less the 52-byte record, so the member region runs from offset $34 to the end of the payload. The checksum is verified by `crc_lib_verify_check_number` before the members are unpacked; it is a plain 32-bit sum of the member-region bytes, described in Chapter 5, not a CRC.

Member Records

From offset $34 the payload is a genuine ZIP archive. It opens with the members as ZIP local file records, each the four-byte signature `50 4B 03 04`, a 26-byte local header carrying the compression method and the compressed and uncompressed sizes, the member name, and then the DEFLATE data. The members are followed by a proper ZIP central directory (one `50 4B 01 02` record per member) and an end-of-central-directory record `50 4B 05 06`. A standard ZIP reader opens the container as-is once the payload is decoded.

This release carries three members: the two hardware-family packages and an installer script. Offsets below are payload-relative, from the start of the decoded payload.

Table 2-4. Members in P39R7 (payload offsets)

Member	Local hdr	Data	Compressed	Uncompressed	CRC-32
`zynq_packet.zip`	$34	$61	25,028,713	25,169,906	$B4FD77AB
`335x_packet.zip`	$17DE8CA	$17DE8F7	15,495,737	15,599,338	$39B5F32F
`update.sh`	$26A5B30	$26A5B57	333	1,138	$4EF70333

The central directory begins at payload offset $26A5CA4 and holds three 46-byte entries plus their names, 285 bytes in all; the end-of-central-directory record follows at $26A5DC1 and closes the payload at $26A5DD7. An earlier revision of this document mistook the third member, the central directory and the EOCD -- 679 bytes together -- for trailing filler, because the cipher error described in Chapter 4 had corrupted them beyond recognition. With the correct cipher they resolve into ordinary ZIP structure and the whole container validates.

Note

The third member, `update.sh`, is the installer the updater runs after unpacking (Chapter 5). It, the central directory and the EOCD all fall inside the first cipher region, which is why a correct cipher is required even to see that the container is a well-formed ZIP.

3

The Obfuscation Transform

The obfuscation is performed by `dev_deconfuse_buff`, which a decoder inverts by applying the same three steps, because each is its own inverse. In the order the function applies them: a full byte reversal, a complement of the second half, and a complement of a sparse set of triangular offsets. The description below is the function read instruction by instruction; a small buffer run through both the function and this description byte-for-byte confirms the reading.

Byte Reversal

The function first reverses the whole payload. It does this in an unusual way that a casual reading mistakes for something cleverer: one loop reverses each half of the buffer in place, and a second loop swaps the two halves. The net effect is a plain full reversal, `payload[::-1]` in one line, and it is equivalent for every length, odd or even, including the payload's odd length. Let `L` be the payload length; then output byte `i` is input byte `L-1-i`.

Second-Half Complement

The function then complements, by exclusive-OR with $FF, every byte in the second half of the reversed buffer. With `H = L // 2`, the complemented range is `[L-H : L)`: the byte at `L-H` and everything after it. On this file `H` is 20,262,635 and the boundary `L-H` is 20,262,636, which falls inside the first member, so the member spans the boundary and a correct decode reproduces it exactly across the seam.

Triangular Complement

Finally the function complements the byte at every triangular offset, that is, at `n(n+1)/2` for `n = 1, 2, 3, ...` while `n(n+1)/2` is less than `L`. These offsets are `1, 3, 6, 10, 15, ...`: nine thousand of them across the forty-megabyte payload, one part in four thousand. Because they are so sparse and so regularly spaced, they defeat a signature scan without materially changing the byte statistics.

Note

The triangular series begins at `n = 1`, so payload offset 0 is never complemented by this step, and it lies outside the second-half range, so it is never complemented at all. An implementation that starts the series at `n = 0` will corrupt the first byte of the container record. It happens not to matter for the members, which begin at $34, but it is wrong.

Why It Looks Encrypted

None of these steps is cryptographic. A chi-square test on the raw file scores 25,219 against a uniform distribution on 255 degrees of freedom, where true ciphertext over forty megabytes would score about 255; the serial correlation is –0.0128 and a Monte-Carlo estimate of pi over the file errs by a tenth of a per cent. These are the signatures of a good compressor's output, seen through a reversal and a sparse mask, not of a cipher. What genuine encryption the file contains is confined to the two regions of Chapter 4.

4

The Triple-DES Layer

The Cipher

The application binary contains a full DES implementation, `Des_Go` at $002C5820, with a two-key triple-DES mode selected by a run-time flag. Its permutation and S-box tables are the standard DES tables: the initial-permutation table at $00631374 matches the DES IP table byte for byte, and the expansion, P-box, S-box, PC-1 and PC-2 tables all match their standard values.

The tables are standard; the algorithm is not. Three details of the round and block plumbing deviate from textbook DES, so a stock DES or 3DES library given the key below will NOT decrypt these regions -- it returns noise. This is the single fact that most obstructs recovery, and it is why earlier work on this format reported the cipher as "implemented the wrong way." The three deviations are:

1. Bytes are converted to bits least-significant-bit first. Where textbook DES numbers the bits of each input byte from the most significant, this implementation numbers them from the least significant, and packs its output bytes the same way.
2. Each S-box's four-bit output is written into the round buffer least-significant-bit first, the reverse of the usual order.
3. The final permutation is applied to the halves as L||R with no closing swap, and decryption is a mirrored round whose active half is L rather than the textbook "same round with the subkeys reversed." Encrypt and decrypt are exact inverses, but the construction is not standard DES.

In triple-DES mode the cipher runs three of these single-DES operations over each eight-byte block in place: the first and third with the first key, the middle one with the second key in the opposite direction -- a two-key EDE arrangement. The update path calls it in the decrypt direction, so a block is recovered as `D(K1, E(K2, D(K1, C)))`. Electronic-codebook mode is used: blocks are independent, with no chaining and no initialisation vector.

The Key and Key Schedule

The 16-byte key is a constant in the binary's data segment at $008A87AC. The key schedule, `the routine at $2C59D4`, clears a 16-byte working key, copies the constant into it, schedules the first eight bytes as the first DES key, and, because the key length is sixteen, schedules the second eight bytes as the second DES key and sets the triple-DES flag.

```
key (16 bytes) = 63 03 21 01 0f 01 00 07 17 10 18 4e 5b 69 37 06

K1 = 63 03 21 01 0f 01 00 07       (first DES key)

K2 = 17 10 18 4e 5b 69 37 06       (second DES key)

mode = two-key triple-DES, EDE, ECB, decrypt
```

Note

The same DES engine and a run-time key are used elsewhere in the firmware to encrypt the instrument's serial-number licence backup on its own filesystem. That path is unrelated to the update format and is not covered here.

The Encrypted Regions

The update decoder, `dev_decode_data`, decrypts exactly two regions of the payload, in place, before handing the buffer to the obfuscation step. Both offsets are relative to the start of the payload, that is, to file offset 112.

Table 4-1. Triple-DES Regions (payload-relative)

Region	Offset	Length	Blocks
1	$00000	$2800 = 10,240	1,280
2	$2E777	$1400 = 5,120	640

Each length is a whole number of eight-byte blocks. The second region begins at an offset that is not a multiple of eight; the cipher nonetheless treats that offset as its first block boundary, and a decoder must do the same.

What the Regions Cover

The two regions sit at the very start of the payload. After the byte reversal, the start of the payload becomes the end of the container, so both regions map to the tail of the decoded container. Region 1, the larger, covers the compressed tail of `335x_packet.zip`, the whole of the third member `update.sh`, and the container's entire central directory and end-of-directory record. Region 2 covers a slab deeper inside `335x_packet.zip`. The first member, `zynq_packet.zip`, occupies the front of the container and never overlaps either region.

Two consequences follow. First, because region 1 lands on the central directory and EOCD, a decode with the wrong cipher does not merely damage one member -- it destroys the ZIP end structure, so the container will not even open as an archive. Getting the cipher right is what turns the tail back into a well-formed ZIP. Second, the cipher still does not touch the current-hardware image: `zynq_packet.zip` lies wholly in front of both regions and decodes correctly from the obfuscation alone. A reader interested only in the Zynq firmware can ignore the cipher; a reader who wants a valid container, the AM3359 image, or the installer needs it.

5

Member Payloads

The Two Packages

The container holds three members: two complete firmware images, one per system-on-chip family, and a short installer script `update.sh`. One .ADS therefore serves both hardware generations from a single file, and the instrument's updater installs whichever package matches the board it is running on. Table 5-1 gives the members' sizes, stored CRC-32 values and, for the two packages, their inner entry counts.

Table 5-1. Members of the Container

Member	Compressed	Uncompressed	CRC-32	Entries
`zynq_packet.zip`	25,028,713	25,169,906	$B4FD77AB	420
`335x_packet.zip`	15,495,737	15,599,338	$39B5F32F	299
`update.sh`	333	1,138	$4EF70333	--

The Zynq-7000 Package

`zynq_packet.zip` is a standard ZIP archive of 420 members holding the complete Zynq-7000 system: the application binary `awg.app`, the root filesystem `rootfs.cramfs`, the boot image, the FPGA bitstream, the Linux kernel `uImage`, the 216 built-in waveform tables, and the driver and configuration trees. It decompresses to 25,169,906 bytes and its stored CRC-32 verifies exactly. This is the member a current SDG2000X installs. Table 5-2 lists its principal members.

Table 5-2. Principal Members of zynq_packet.zip

Member	Size	Content
`awg.app`	12,489,120	Main application. Stripped ARM EABI5 ELF, dynamically linked.
`rootfs.cramfs`	10,858,496	Root filesystem, cramfs v2, 2,102 files, BusyBox based.
`BOOT.bin`	4,588,024	Zynq first-stage boot image.
`config/fpga/top_unicorn_zynq.bit`	4,045,674	FPGA bitstream.
`uImage`	3,132,776	Linux kernel image.
`devicetree.dtb`	15,724	Flattened device tree.
`config/arb/*.bin`	varies	216 built-in waveform tables, all but three of 32,768 bytes.
`drivers/*.ko`	varies	siglent_vdma, xilinx_axidma, g_usbtmc, gpib, siglent_touch_irq.

The application binary is a stripped ARM EABI5 ELF, dynamically linked against the BusyBox-based root filesystem. Its symbol names survive in run-time type information and in assertion strings, and the build path `/home/brus_peng/workdir/sdg2000x_zynq/trunk/sdg/lib/src/` appears throughout, which identifies the binary as the SDG2000X Zynq build rather than one of the other families the source tree serves.

The same source tree evidently builds the whole SDG line. Type names recovered from the binary include driver classes for the SDG1000, SDG2000, SDG6000 and SDS2000X Plus. This shared lineage is why the binary carries, parses and then discards parameters that belong to other models.

The AM3359 Package

`335x_packet.zip` is the corresponding archive for the earlier hardware, built around the Texas Instruments Sitara AM3359 processor rather than the Zynq. It is itself a ZIP of 299 entries: a boot script, the `MLO` second-stage bootloader, a U-Boot environment, and the AM3359 application `sdg2000.app`, a 12.9-megabyte ARM ELF. It decompresses to 15,599,338 bytes and, with the corrected cipher, every one of its entries -- including `sdg2000.app` -- inflates bit-exact and CRC-valid.

One .ADS therefore serves both hardware generations from a single file; the instrument installs whichever member matches the board it is running on. On a Zynq instrument the AM3359 member is carried but not installed.

Installation

The container's third member is a top-level `update.sh` (a POSIX shell script that begins `#! /bin/sh`); the updater runs it after unpacking to drive the install. Each hardware package in turn carries its own installer, `zynq_update.sh`, which writes the boot image and kernel to raw NAND partitions and then copies the application and configuration files into place.

```
flash_erase /dev/mtd<n> 0 0

nandwrite -s 0 -p /dev/mtd<n> /usr/bin/siglent/usr/usr/upgrade/<uImage>

cp -pf  /usr/bin/siglent/usr/usr/upgrade/*.app  /usr/bin/siglent/

cp -rpf /usr/bin/siglent/usr/usr/upgrade/bin    /usr/bin/siglent/
```

Warning

The kernel and boot image are written by erasing a raw NAND partition and writing it again. An interruption between the erase and the write leaves the instrument without a bootable image, which is why the manufacturer's update instructions say not to remove power during an update. There is no recovery path over the remote interface.

Integrity Checks

The format carries integrity checks at two levels. Each ZIP member, inner and outer, has the usual stored CRC-32 of its uncompressed data. Above them, the container record of Chapter 2 carries a checksum over the whole member region, verified before anything is unpacked.

The container checksum is not a CRC. The routine `crc_lib_get_check_sum` that computes it is a plain running sum: it adds every byte of the region into a 32-bit accumulator and stores the result. A decoder that wants to reproduce it sums the bytes from payload offset $34 for the length in the record's length field.

6

Reference Decoder

The Decode Pipeline

To recover the members from a .ADS file, in order:

1. Take the payload as the file from offset 112 to the end.
2. Decrypt payload region $00000 for $2800 bytes, and region $2E777 for $1400 bytes, each with the non-standard two-key triple-DES of Chapter 4 in ECB decrypt mode. A stock DES library will not do; the three inner-loop deviations must be reproduced.
3. Reverse the whole payload.
4. Complement (XOR $FF) every byte from offset `L - L//2` to the end.
5. Complement every byte at a triangular offset `n(n+1)/2` for `n = 1, 2, ...` below `L`.
6. From offset $34, open the payload as an ordinary ZIP archive and inflate each member (recursing into the two nested ZIP packages).

Python Implementation

The following decoder is complete and dependency-free: it uses only the standard library, and carries the non-standard cipher itself because no stock library implements it. The three deviations of Chapter 4 are marked. It reproduces the results in Chapter 7. (Standard DES tables are elided here for space; the shipped `ads_decode_full.py` carries them in full.)

```
import struct, io, zipfile, zlib
IP,FP,E,P,PC1,PC2,SHIFT,SBOX = _standard_des_tables()  # see ads_decode_full.py
KEY = bytes.fromhex('630321010f0100071710184e5b693706')
K1, K2 = KEY[:8], KEY[8:16]
REGIONS = [(0x0, 0x2800), (0x2e777, 0x1400)]

def perm(b, t): return [b[i-1] for i in t]
def b2b(bs):                              # (1) bytes->bits, LSB-first
    return [(by >> i) & 1 for by in bs for i in range(8)]
def b2s(bits):                            # (1) bits->bytes, LSB-first
    return bytes(sum(bit << j for j, bit in enumerate(bits[i:i+8]))
                 for i in range(0, len(bits), 8))
def subkeys(k):
    k = perm(b2b(k), PC1); C, D = k[:28], k[28:]; ks = []
    for s in SHIFT:
        C = C[s:]+C[:s]; D = D[s:]+D[:s]; ks.append(perm(C+D, PC2))
    return ks
def sbox(x):                              # (2) S-box output, LSB-first
    o = []
    for i in range(8):
        b = x[i*6:i*6+6]; r = (b[0]<<1)|b[5]
        c = (b[1]<<3)|(b[2]<<2)|(b[3]<<1)|b[4]; v = SBOX[i][r*16+c]
        o += [v&1, (v>>1)&1, (v>>2)&1, (v>>3)&1]
    return perm(o, P)
def enc(b8, ks):                          # active half = R
    x = perm(b2b(b8), IP); L, R = x[:32], x[32:]
    for k in ks:
        L, R = R, [a^b for a,b in zip(L, sbox([a^b for a,b in zip(perm(R,E),k)]))]
    return b2s(perm(L+R, FP))             # (3) FP over L||R, no closing swap
def dec(b8, ks):                          # (3) mirrored round, active half = L
    x = perm(b2b(b8), IP); L, R = x[:32], x[32:]
    for k in reversed(ks):
        L, R = [a^b for a,b in zip(sbox([a^b for a,b in zip(perm(L,E),k)]), R)], L
    return b2s(perm(L+R, FP))
```

6-2 Reference Decoder

```
def des3(d):                              # 2-key EDE decrypt: D(K1,E(K2,D(K1,x)))
    a, b = subkeys(K1), subkeys(K2)
    return b''.join(dec(enc(dec(d[i:i+8], a), b), a)
                    for i in range(0, len(d)//8*8, 8))


def decode(ads):
    p = bytearray(ads[112:])              # payload = file[112:]
    for off, ln in REGIONS:               # decrypt the two regions in place
        n = ln // 8 * 8; p[off:off+n] = des3(bytes(p[off:off+n]))
    b = bytearray(p)[::-1]; L = len(b); H = L // 2
    for i in range(L-H, L): b[i] ^= 0xFF   # complement second half
    n = 1
    while n*(n+1)//2 < L:                  # complement triangular offsets
        b[n*(n+1)//2] ^= 0xFF; n += 1
    return bytes(b)

st = decode(open('SDG2000X_P39R7.ADS','rb').read())
zf = zipfile.ZipFile(io.BytesIO(st[st.find(b'PK\x03\x04'):]))  # a real ZIP
```

7

Recovery and Verification

How the Format Was Recovered

The container transforms were recovered first, from the file alone, by recognising the reversal and fitting the two masks so that the first member inflated and its CRC verified. That gave the Zynq package in full and left the AM3359 package failing partway through, which is what led to the cipher.

The cipher was then read from the binary rather than guessed. The update routines were located through the dynamic symbol table, `dev_update_system` and `dev_upgrade_detail_acitve` and `dev_decode_data` read in turn, and the key and regions taken from the constants those routines pass. The DES tables in `Des_Go` matched the standard tables, which at first suggested a standard cipher -- but a standard-DES decrypt of the regions returned noise. Reading the round and block routines instruction by instruction exposed the three deviations of Chapter 4: LSB-first bit ordering, an LSB-first S-box output, and a non-standard final permutation with a mirrored decrypt. A dependency-free implementation of exactly those steps decrypts the header and both regions, and agrees byte-for-byte with the community's independently derived `pyDesSiglent.py`. The obfuscation was likewise confirmed by reading `dev_deconfuse_buff` instruction by instruction rather than by fitting.

Verification Results

The decoder of Chapter 6 was run against two releases, the current P39R7 and the earliest available P17R5. Every member of both, at every level of nesting, inflates and verifies its stored CRC-32 exactly, and the decrypted header's length field matches the decoded payload length in each.

Table 7-1. Decode Results

Check	P39R7	P17R5
Header length field = decoded length	yes ($26A5DD7)	yes ($C547F5)
Container opens as a valid ZIP	yes, 3 members	yes, flat
`zynq_packet.zip` (420 entries)	all CRC-exact	n/a
`335x_packet.zip` (299 entries)	all CRC-exact	n/a
`sdg2000.app` (12.9 MB ELF)	bit-exact	n/a
Total files, bad CRCs	679, 0	272, 0

Both packages, and the AM3359 application that earlier stopped short, are now recovered bit-for-bit and verified against their own checksums. The container opens directly in any ZIP tool once the payload is decoded. There is no residual.

The Former AM3359 Residual

An earlier revision of this document reported that the AM3359 application recovered to within 733 bytes of its stated size and no further, and attributed the shortfall to a structural alignment inconsistency in Siglent's packaging. That conclusion was wrong, and is corrected here.

The shortfall was an artifact of the cipher, not of the format. The earlier decoder used a stock triple-DES library, on the reasonable but mistaken assumption that standard DES tables meant a standard cipher. Because the two encrypted regions land on the tail of the container -- the end of `335x_packet.zip`, the whole of `update.sh`, and the central directory and EOCD -- a wrong cipher corrupts exactly that tail. The visible symptom was a compressed stream that ended early and a container with no readable directory; the 733-byte figure was simply how far into the last member the corruption happened to begin.

With the non-standard cipher of Chapter 4 the same regions decrypt correctly, the tail of `335x_packet.zip` completes, `update.sh` and the central directory reappear, and `sdg2000.app` inflates to its full 12,896,400 bytes with a matching CRC-32. The section-header table that was thought lost is present. The same holds in P17R5, whose corresponding member also now verifies. Nothing about the format is undecoded.

Note

The lesson worth recording: standard permutation and S-box tables do not imply a standard cipher. The three inner-loop deviations of Chapter 4 are invisible in the tables and are only found by reading the round logic. This is the trap that gave the format a reputation for being only partly recoverable; it is fully recoverable.

8

Field Reference

File and Payload

`File header`	Bytes $0000 to $006F, 112 bytes. Encrypted (Chapter 4), not obfuscated. Decrypts to: file CRC (u32 $00), decoded length (u32 $04), product id (u32 $0C), vendor tag `SIGLENT` ($26), USB host-controller tag `ISP1763` ($3A). Consumed by `dev_update_system` for the pre-flight check; not part of the container.
`Payload`	Bytes $0070 to end. All transforms operate here. Length on P39R7 is 40,525,271 ($26A5DD7).

Container Record (payload offset $00)

`$00 checksum (u32)`	32-bit sum of the member-region bytes. P39R7: $C3F3F2A3.
`$04 length (u32)`	Member-region length, payload minus 52. P39R7: $026A5DA3 = 40,525,219.
`$08 type (u8)`	Type code. P39R7: 7.
`$09 to $33`	Reserved, zero.
`$34 onward`	Member records (ZIP local files), back to back.

ZIP Local File Header (per member)

`$00 signature`	`50 4B 03 04`.
`$08 method (u16)`	8 = deflate.
`$0E crc-32 (u32)`	CRC of the uncompressed member.
`$12 comp size (u32)`	Compressed length; distance to the next member.
`$16 uncomp size (u32)`	Uncompressed length.
`$1A name len / $1C extra len (u16)`	Header extends by these before the data.

9

Constants

Cipher

`Algorithm`	Non-standard two-key triple-DES, EDE, ECB, decrypt direction. Standard DES tables, three inner-loop deviations (Chapter 4): LSB-first byte/bit conversion, LSB-first S-box output, and a final permutation over L‖R with a mirrored decrypt. A stock DES/3DES library will not decrypt these regions.
`Key (16 bytes)`	`63 03 21 01 0F 01 00 07 17 10 18 4E 5B 69 37 06`
`As 24-byte EDE`	`K1 || K2 || K1`
`Region 1`	payload $00000, length $2800 (10,240 bytes).
`Region 2`	payload $2E777, length $1400 (5,120 bytes).
`Key constant`	application binary data segment $008A87AC.
`DES engine`	`Des_Go` at $002C5820; standard IP table at $00631374.
`Cross-check`	agrees byte-for-byte with the community `pyDesSiglent.py` (EEVblog thread).

Obfuscation

`Step 1`	Full byte reversal of the payload.
`Step 2`	Complement $FF from offset `L - L//2` to end.
`Step 3`	Complement $FF at `n(n+1)/2`, `n >= 1`, below `L`.

Firmware Routines

`dev_update_system`	Drives the update; reads and checks the header.
`dev_decode_header_data`	Decrypts the 112-byte header.
`dev_upgrade_detail_acitve`	Reads the payload, verifies the checksum, decrypts and de-obfuscates.
`dev_decode_data`	Decrypts the two cipher regions, then calls the de-obfuscator.
`dev_deconfuse_buff`	The obfuscation: reverse, complement, triangular.
`Des_Go / $2C59D4`	The DES engine and its key schedule.
`crc_lib_get_check_sum`	The container checksum (a byte sum).